Nicholas Barkero

Professor Damavandi

Project 1 Report

16 October 2026

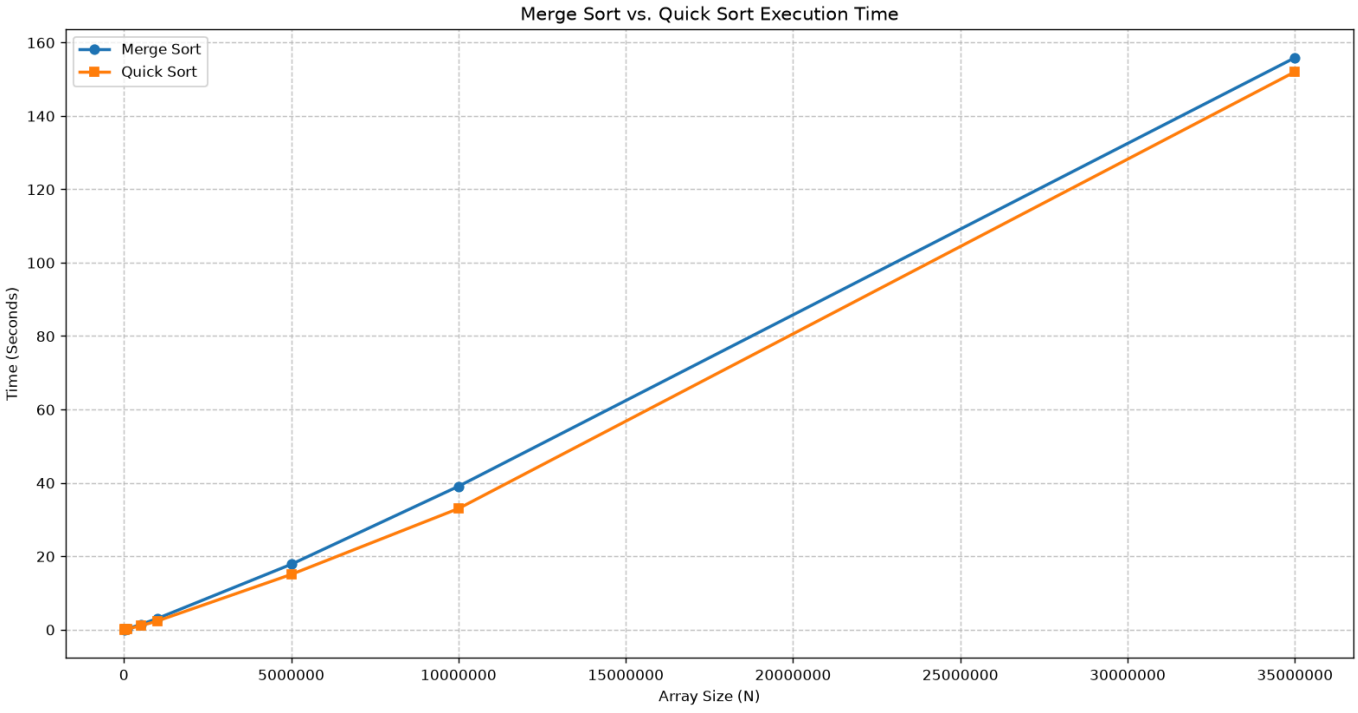
Project 1 Report

Quick Sort and Merge Sort:

Time Complexities:

Merge Sort: O(N log N) in all cases (best, average, worst) because the array is always divided in half and merged linearly.

Quick Sort: O(N log N) in the best and average cases, but goes to O(N^2) in the worst case (when the array is already sorted or contains a lot of duplicates, or a poor pivot is chosen).



Both were equally fast starting off, but Quick sort, as n increased, was always a little faster.

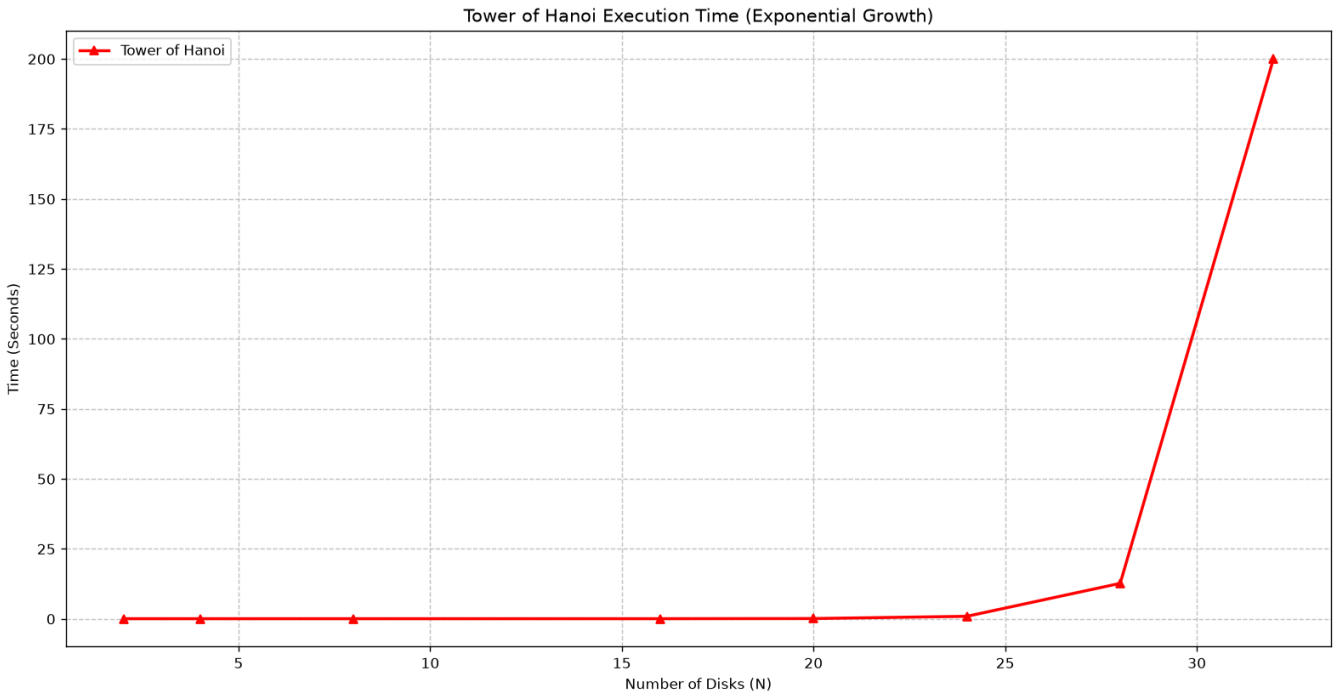
Tower of Hanoi:

Recurrence Relation:

The algorithm requires moving n-1 disks, then moving the largest disk, then moving the n-1 disks again. This gives the relation: $T(n) = 2T(n-1) + 1$.

Time Complexity:

Solving the recurrence relation yields $2^n - 1$ moves, making the time complexity strictly exponential: $O(2^n)$.

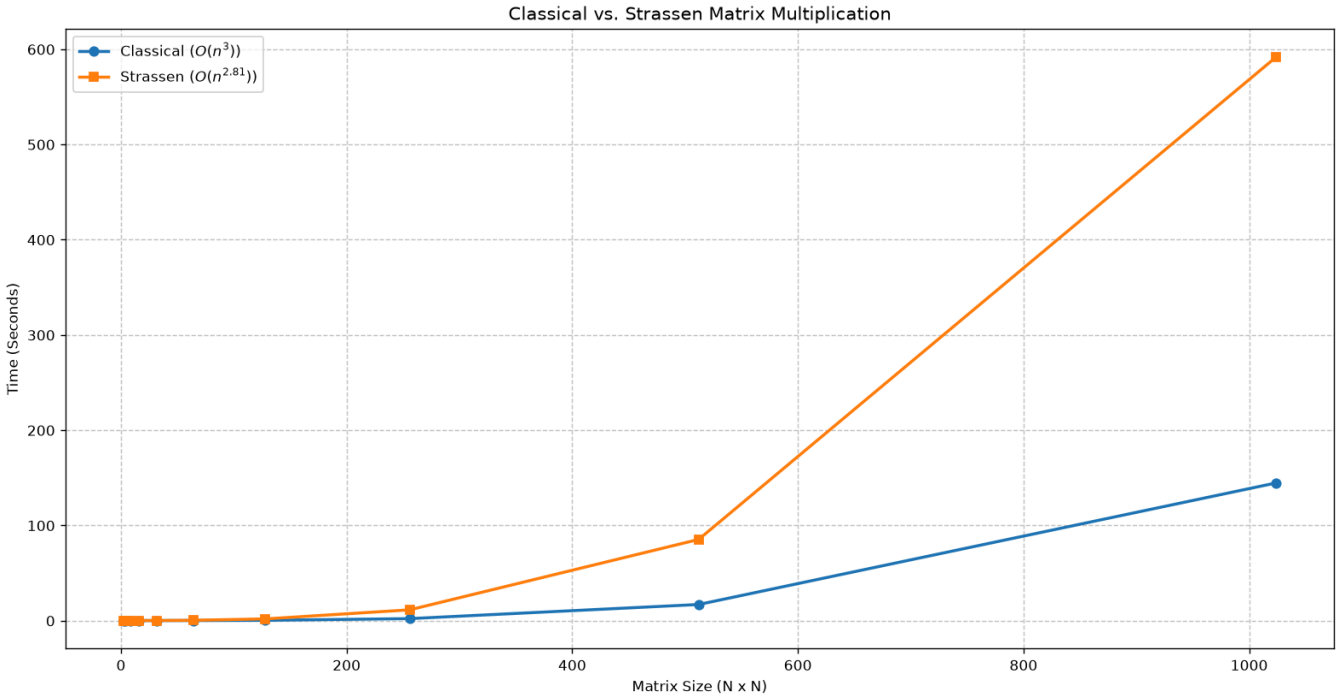


Matrix Multiplication:

Time Complexities:

Classical Multiplication: O(n^3). This is due to the three nested for loops required to calculate the dot product of every row and column.

Strassen's Algorithm: O($n^{log_2 7}$) = O(n^2.81). By utilizing a divide-and-conquer approach, Strassen's algorithm reduces the number of recursive multiplications from 8 to 7 per step.



They were both relative until around the 250 matrix size, where the gap between Classical and Strassen were shown. Classical, at higher n's, was by far superior in terms of time complexity.

Difficulties and Solutions:

A recursion error would occur with Quick Sort with n elements going past 10,000,000. This happened when partitioning because the original algorithm used a pivot which was always the last element. The solution was to randomize the pivot and swap the pivot with the last element.

The exponential nature of the Tower of Hanoi was O(2^n) causing execution time to increase a lot, especially from n=16 to n=32. It ended up almost always passing 10 minutes by this point. The solution was to stop it at the 120 second mark because if it took that long for the previous n to complete, the next n would take longer than 10 minutes.

Matrices at 512 or 1024 became too large and went past 10 minutes. The solution was to simply stop the execution at 120 seconds before test n size that would on average take 8 or more minutes.